Reviewer Pack — Minimal Symmetry Invariant for Tseitin Resolution Length

Entry d7e4a320cb95 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 16:24:14 UTC

Minimal Symmetry Invariant for Tseitin Resolution Length

Entry ID: `d7e4a320cb95` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 16:24:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Resolution Proof Complexity

Statement:

> ['For every Tseitin formula, there exists a polynomial-time computable symmetry invariant such that the Resolution refutation length of the formula is at least exponential in the size of the smallest non-trivial orbit under this symmetry.', 'Formally, for any Tseitin formula F with n variables and m clauses, there exists an algebraic invariant ν(F) such that the Resolution proof complexity of F satisfies: ResolutionProofComplexity(F) ≥ 2^(Ω(ν(F)))', 'Moreover, for all Tseitin formulas G where ν(G) = ν(F), the Resolution proof complexities satisfy: ResolutionProofComplexity(G) = Θ(ResolutionProofComplexity(F)).']

Rationale (proposer's reasoning):

> ['Algebraic invariants from combinatorial group theory and representation theory have been used to study circuit complexity. This conjecture explores the use of symmetry invariants, which are less commonly applied in complexity theory, to provide lower bounds on resolution proof length for Tseitin formulas.', 'By leveraging the structure provided by symmetries, it is expected that these invariants can expose new ways to understand the inherent hardness of certain types of formulas.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `873f8abd0e0b5335`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Tseitin formula F, if the Resolution refutation length is at least exponential (ResolutionProofComplexity(F) ≥ 2^(10n)) in the size of its smallest non-trivial orbit, or two formulas G and H with the same symmetry invariant ν(G) = ν(H) have comparable resolution proof complexities (ResolutionProofComplexity(G) = Θ(ResolutionProofComplexity(H))) for n variables, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Tseitin formula" AND "symmetry invariant" AND "algebraic combinatorics" - "Resolution proof complexity" AND "polynomial-time computable invariant" AND "Tseitin formula" - "exponential lower bound" AND "resolution refutation length" AND "algebraic invariant"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clauses.append([variables[i-1]])
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clauses.append([f'~{variables[i-1]}', variables[j-1]])
                clauses.append([f'~{variables[j-1]}', variables[i-1]])
        return variables, clauses

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                raise ValueError("Matrix is singular")
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
```

```
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def compute_symmetry_invariant(variables, clauses):
    n = len(variables)
    G = [[0] * n for _ in range(n)]
    for clause in clauses:
        if len(clause) == 1:
            var = clause[0]
            i = int(var[1:]) - 1
            G[i][i] = 1
        elif len(clause) == 2:
            var1, var2 = clause[0], clause[1]
            i = int(var1[1:]) - 1
            j = int(var2[1:]) - 1
            G[i][j] = G[j][i] = 1
    return sum(sum(row) for row in G)

def resolution_proof_complexity(clauses):
    m = len(clauses)
    A = [[0] * (m+1) for _ in range(m+1)]
    for i in range(m):
        for j in range(i, m):
            if clauses[i][0] == f'~{clauses[j][0]}':
                A[i][j] = A[j][i] = 1
    rank = gaussian_elimination(A)
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    variables, clauses = generate_tseitin_formula(n)
    invariant = compute_symmetry_invariant(variables, clauses)
    proof_complexity = resolution_proof_complexity(clauses)
    results.append({
        "n": n,
        "invariant": invariant,
        "proof_complexity": proof_complexity
    })

mean_complexity = sum(result["proof_complexity"] for result in results) / len(results)
std_complexity = math.sqrt(sum((result["proof_complexity"] - mean_complexity) ** 2 for result

conjecture_holds = all(result["proof_complexity"] >= 2**(10 * n_values[0]) for result in resul
counterexample = "" if conjecture_holds else "mapping_undefined"
```

```
    return {
        "metric_name": "Resolution Proof Complexity",
        "metric_value": mean_complexity,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_complexity = sum(result["metric_value"] for result in results) / len(results)
    std_complexity = math.sqrt(sum((result["metric_value"] - mean_complexity) ** 2 for result in r
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_complexity} std={std_complexity} support_fraction={su
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b66acbb7.py", line 115, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b66acbb7.py", line 87, in run_trial
    invariant = compute_symmetry_invariant(variables, clauses)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b66acbb7.py", line 67, in compute_symme
    i = int(var1[1:]) - 1
        ^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'x1'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

The test code crashed before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Re-run the test with proper error handling and ensure that it produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15934
2	preregistration	ollama_remote	glm4:latest	0	0	9711
3	novelty	ollama_remote	glm4:latest	0	0	8854
4	novelty	ollama_remote	glm4:latest	0	0	8767
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19001
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8150
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8437
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14393
9	judge	ollama_remote	glm4:latest	0	0	11406

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 104653 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d7e4a320cb95.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d7e4a320cb95.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d7e4a320cb95.tar.gz` (if generated)